

Deadlocks

- Deadlocks is a fundamental problem in systems.
- A process may request resources in any order, which may not be known a priori and a process can request resource while holding others.
- If the sequence of the allocations of resources to the processes is not controlled, deadlocks can occur.
- A deadlock is a state where a set of processes request resources that are held by other processes in the set.

System Model

- System consists of resources
- Resource types $R_1, R_2, \dots, R_m$
CPU cycles, memory space, I/O devices
- Each resource type R_i has W_i instances.
- Each process utilizes a resource as follows:
 - **request**
 - **use**
 - **release**

Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

- **Mutual exclusion:** only one process at a time can use a resource
- **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes
- **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task
- **Circular wait:** there exists a set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 , ..., P_{n-1} is waiting for a resource that is held by P_n , and P_n is waiting for a resource that is held by P_0 .

Deadlock Characterization

Possible strategies

- **Mutual exclusion**: allow shared resources to be used with ME.
- **Hold and wait**: Let a process start only after it receives all resources it requires (starvation) OR acquire only when you need it.
- **No preemption**: A process should wait only if the resources it needs is not available and they are not in possession of another waiting process. If being used by another waiting process then grab them and see if you can run. You wait only if the resource is with a running process. While waiting some of your resources might get grabbed
- **Circular wait**: Total ordering of resources so won't be cycle. Each process attains resources in order as in the total ordering.

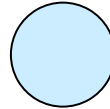
Resource-Allocation Graph

A set of vertices V and a set of edges
 E .

- V is partitioned into two types:
 - $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the processes in the system
 - $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system
- **request edge** – directed edge $P_i \rightarrow R_j$
- **assignment edge** – directed edge $R_j \rightarrow P_i$

Resource-Allocation Graph (Cont.)

- Process

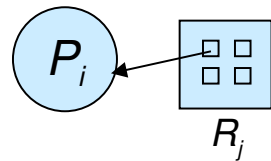
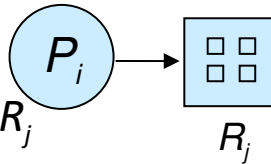


- Resource Type with 4 instances

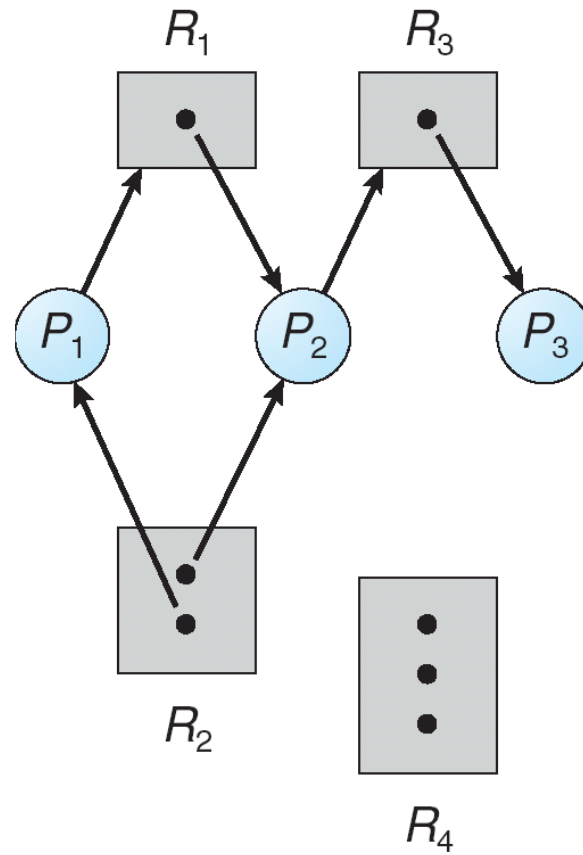


- P_i requests instance of R_j

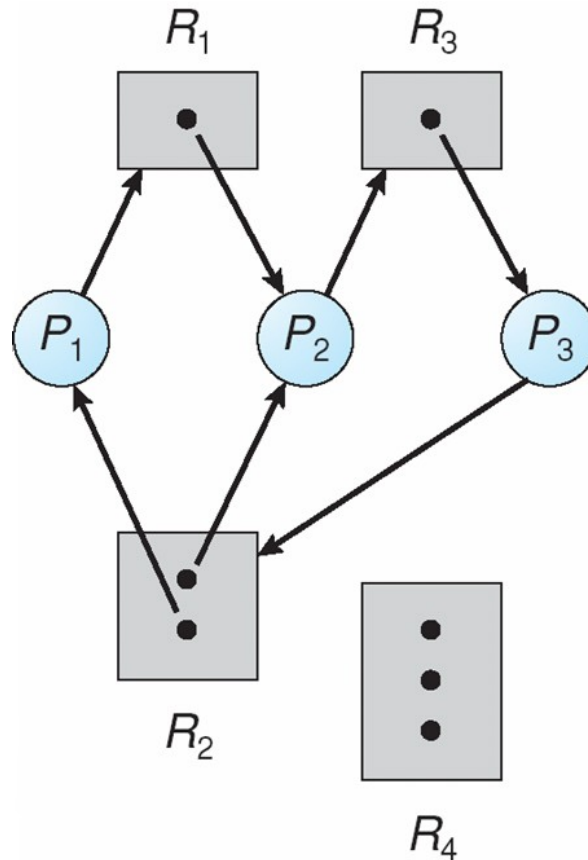
- P_i is holding an instance of R_j



Example of a Resource Allocation Graph

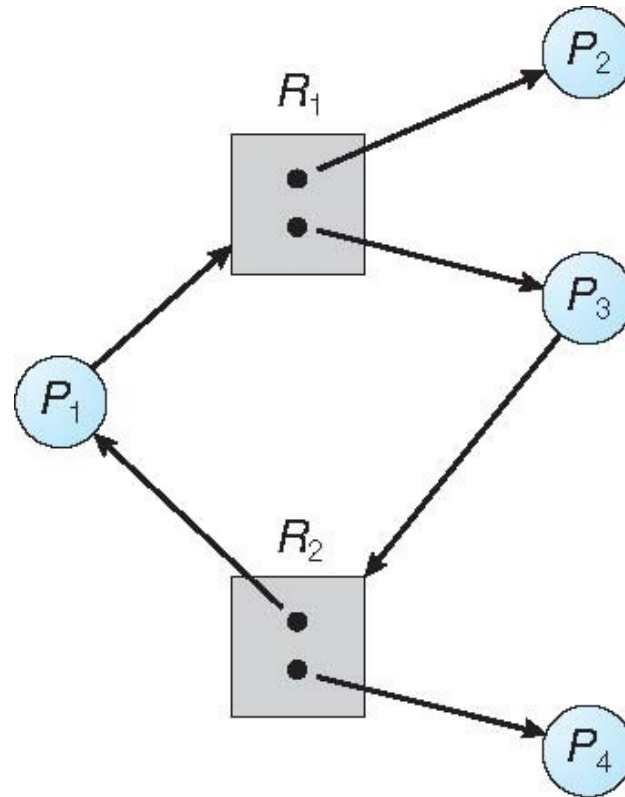


Resource Allocation Graph With A Deadlock



Is there a deadlock?
How can we detect
a deadlock?
Is it a iff condition?

Graph With A Cycle But No Deadlock



Basic Facts

- If graph contains no cycles $\Rightarrow$ no deadlock
- If graph contains a cycle $\Rightarrow$
 - if only one instance per resource type, then deadlock
 - if several instances per resource type, possibility of deadlock

Methods for Handling Deadlocks

- Ensure that the system will *never* enter a deadlock state:
 - Deadlock prevention
 - Deadlock avoidance
- Allow the system to enter a deadlock state and then recover
- Ignore the problem and pretend that deadlocks never occur in the system; used by most operating systems, including UNIX

Deadlocks

- The problem appears in standard systems too.
- An Operating System can work on this problem at multiple levels:
 - Detect deadlocks
 - Recover from deadlocks
 - Prevent deadlocks
- Various algorithms exist for these problems.

To The Distributed Setting

- There is no physical global clock in the system to which processes have instantaneous access.
- The communication medium may deliver messages out of order, messages may be lost garbled or duplicated due to timeout and retransmission, processors may fail and communication links may go down.

To The Distributed Setting

- We make the following assumptions:
 - The systems have only reusable resources.
 - Processes are allowed to make only exclusive access to resources.
 - **There is only one copy of each resource**
- Hence a system is deadlocked iff there exists a cycle or knot in WFG

Resource Request Models

The manner in which a task asks for resources can be

Single Resource request Model: can have at most one pending resource request

AND Resource request Model: Can request multiple resources. Task resumed only when all are acquired.

Example: (R1 AND R2 AND R3)

OR Resource request Model: Can request multiple resources. Task can resume when any one of the request resources becomes available

Example: (R1 *or* R2) or (R1 *or* R2 *or* R3)

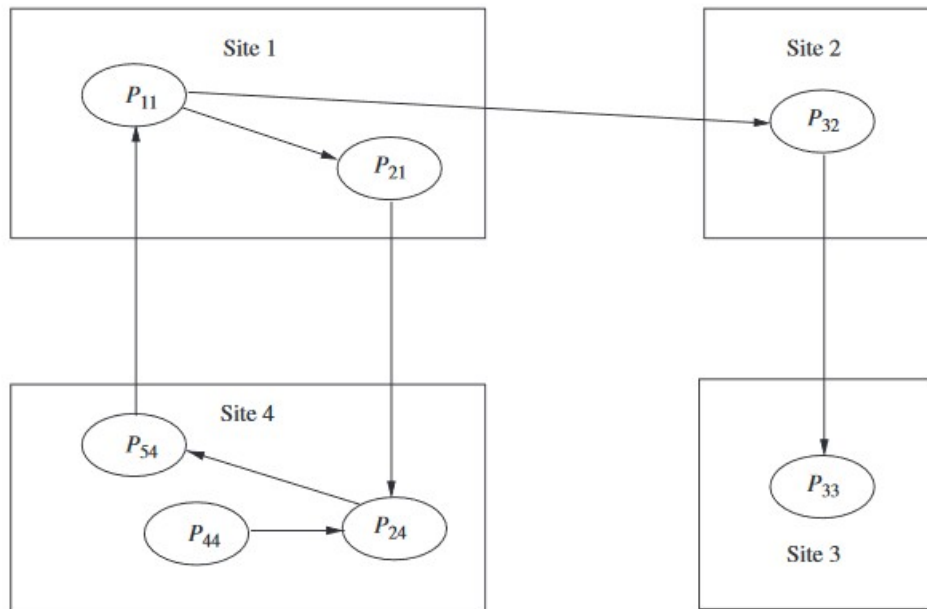
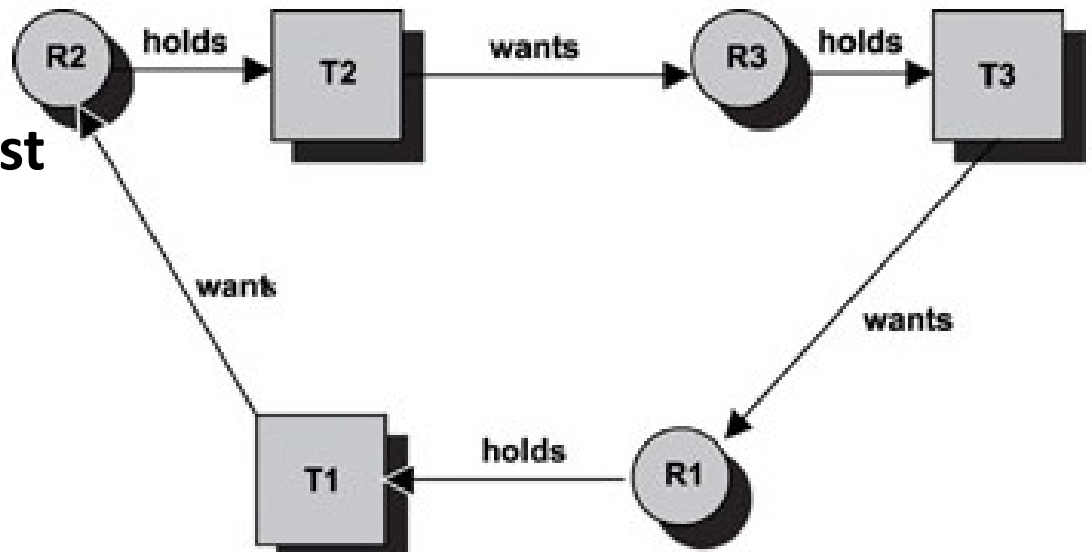
AND OR Resource request Model: Any combination of AND and OR.

Example: X AND (Y OR Z)

detecting using WFG does not directly work but algorithms work on doing repeated OR tests over time (as deadlocks don't go away)

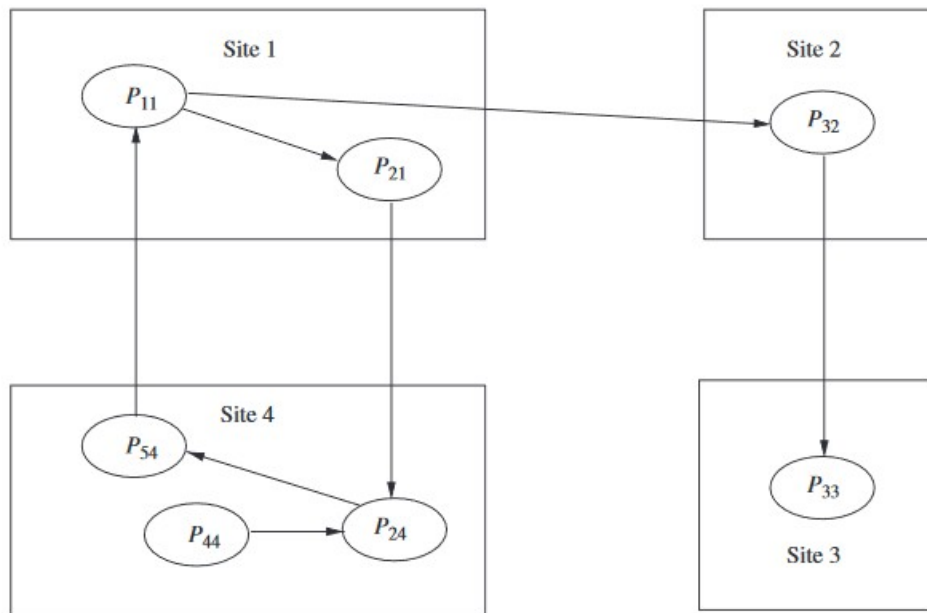
Single Resource Request Model

Presence of cycle indicates deadlock?



AND Resource request Model

If there is a cycle then there is a deadlock? there is a process deadlocked then will it be part of a cycle?

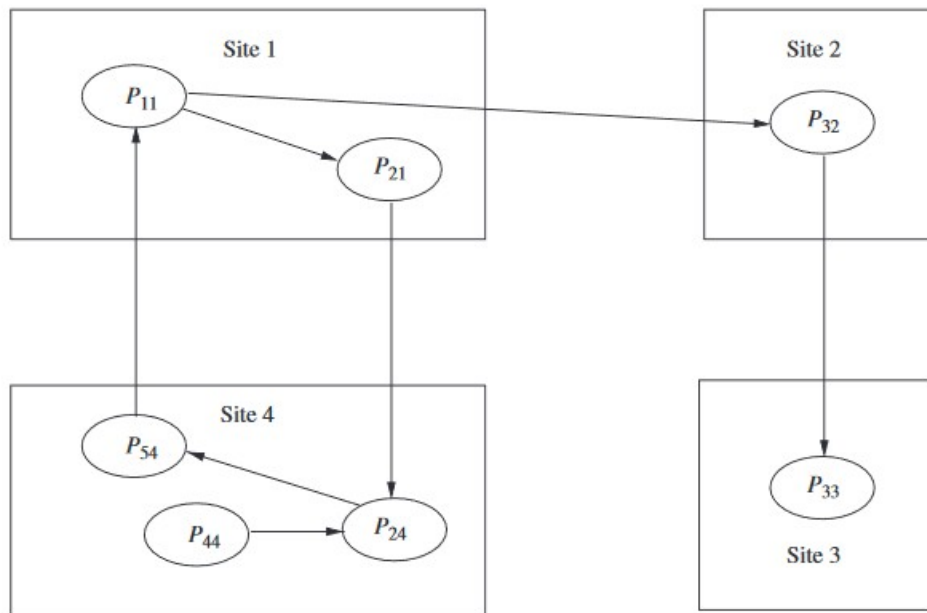


OR Resource request Model

If there is a cycle then there is a deadlock?

A knot(K) consists of a set of nodes such that for every node a in K , all nodes in K and only the nodes in K are reachable from node a .
(SCC with no outgoing edge?)

A knot indicates a deadlock in the OR Model.
Are processes in knot alone deadlocked?



OR Resource request Model

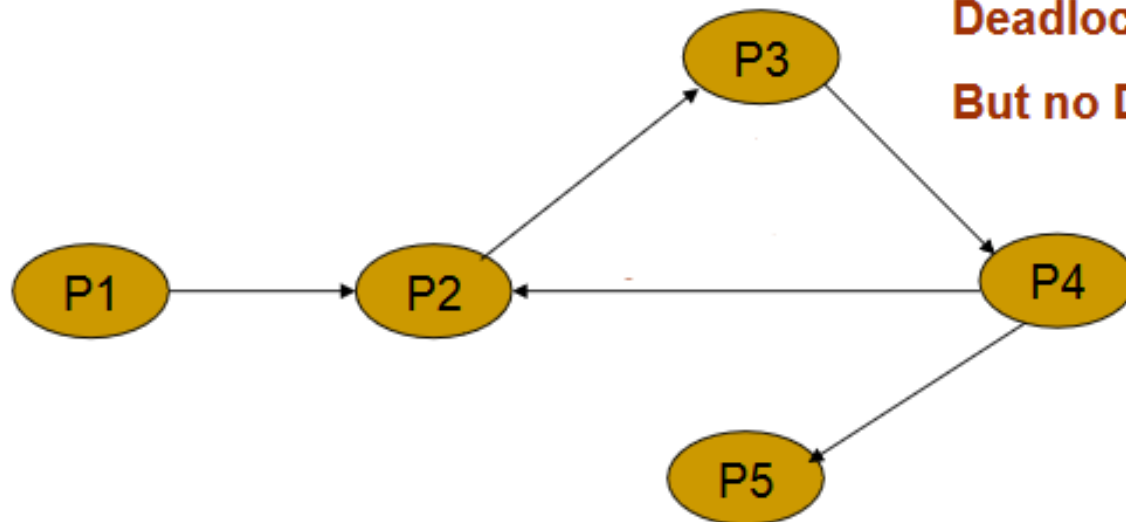
If there is a cycle then there is a deadlock?

No, P_{33} , then P_{32} and then P_{11} will run

A knot(K) consists of a set of nodes such that for every node a in K , all nodes in K and only the nodes in K are reachable from node a .
(SCC with no outgoing edge?)

A knot indicates a deadlock in the OR Model.

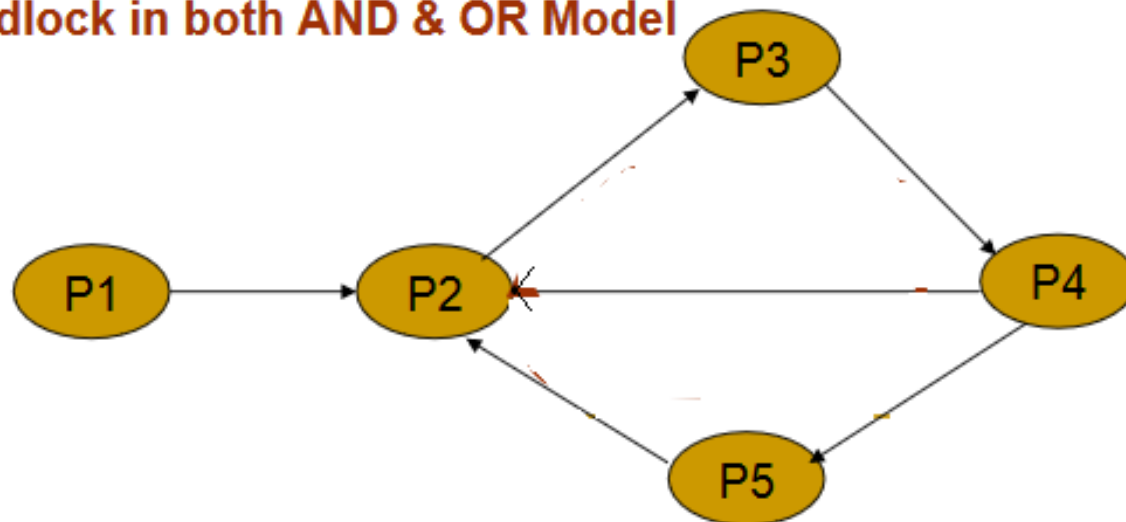
Are processes in knot alone deadlocked? No.. P_{44} too is



Deadlock in AND Model
But no Deadlock in OR Model

Cycle but no Knot

Deadlock in both AND & OR Model



Cycle & Knot

Deadlock conditions

- The condition for deadlock in a system using the AND condition is the existence of a *cycle*.
- The condition for deadlock in a system using the OR condition is the existence of a *knot*.

Hence a system is deadlocked iff there exists a
cycle or knot in WFG

Centralized Deadlock-Detection Algorithms

- Ask every process about its wait for status and create a global wait for graph. Will this work?

Centralized Deadlock-Detection Algorithms

- The Ho-Ramamoorthy Algorithms

- The Two-Phase Algorithm

Deadlocks don't go away unless broken. Collect status twice. Take only those dependencies which are present in both. Any issues?

- The One-phase Algorithm

Uses process status table and resource status table .
process status table contains resources that are locked or waited on for each process. resource status table contains processes using or waiting on each resource. Takes a dependency as valid only if both tables indicate it.

Centralized Deadlock-Detection Algorithms

- The One-phase Algorithm

process status table: the process' information where for each process gives information about resources held or sought after.

Resource status table: Processes to which the resource is allocated to or seeking it

Gather this from all sites and create WFG if tables gathered from both sites say the same information. That is site 1 says P waits for R which is on site 2. Site2 also says P requested for R

Can we have false deadlock?

Centralized Deadlock-Detection Algorithms

- Consider a false cycle by $P1(\text{site1})$, $R1(\text{site 2})$, $P2(\text{site 3})$ and $R2(\text{site 4})$ that the two phase algorithm detected where at
 - At site2 we had $P1 \rightarrow R1 \rightarrow P2$ and
 - At site4 we had $P2 \rightarrow R2 \rightarrow P1$

When the information from site1 is collected it says either $P1 \rightarrow R1$ or $R2 \rightarrow P1$. Since it won't say both you cannot construct a cycle though site 2 and site 4 might indicate $P1 \rightarrow R1 \rightarrow P2$ and $P2 \rightarrow R2 \rightarrow P1$

Distributed Edge-Chasing Algorithm

Chandy Misra Haas Algorithm

- A probe is sent by a blocked

process to the process holding the resource.

- Probe is a triplet (I, J, k)

I: ID of blocked process

J: ID of process sending the message

K: ID to which message is sent

- If probe is received by a blocked process, it edits and forwards the probe to the processes holding the resource it requires
- If initial blocked process receives back its probe; it declares deadlock

Distributed Edge-Chasing Algorithm

Chandy Misra Haas Algorithm

To determine if a blocked process is deadlocked

if P_i is locally dependent on itself

then **declare a deadlock**

else for all P_j and P_k such that

(a) P_i is locally dependent upon P_j , and

(b) P_j is waiting on P_k , and

(c) P_j and P_k are on different sites,

send *probe* (i, j, k) to the home site of P_k

On the receipt of *probe* (i, j, k), the site takes the foll. actions:

if (a) P_k is blocked, and

(b) $dependent_k(i)$ is false, and // P_k does not know of i 's dependency on it.

(c) P_k has not replied to all requests of P_j ,

then begin

$dependent_k(i) = \text{true};$

if $k = i$ then declare that P_i is deadlocked

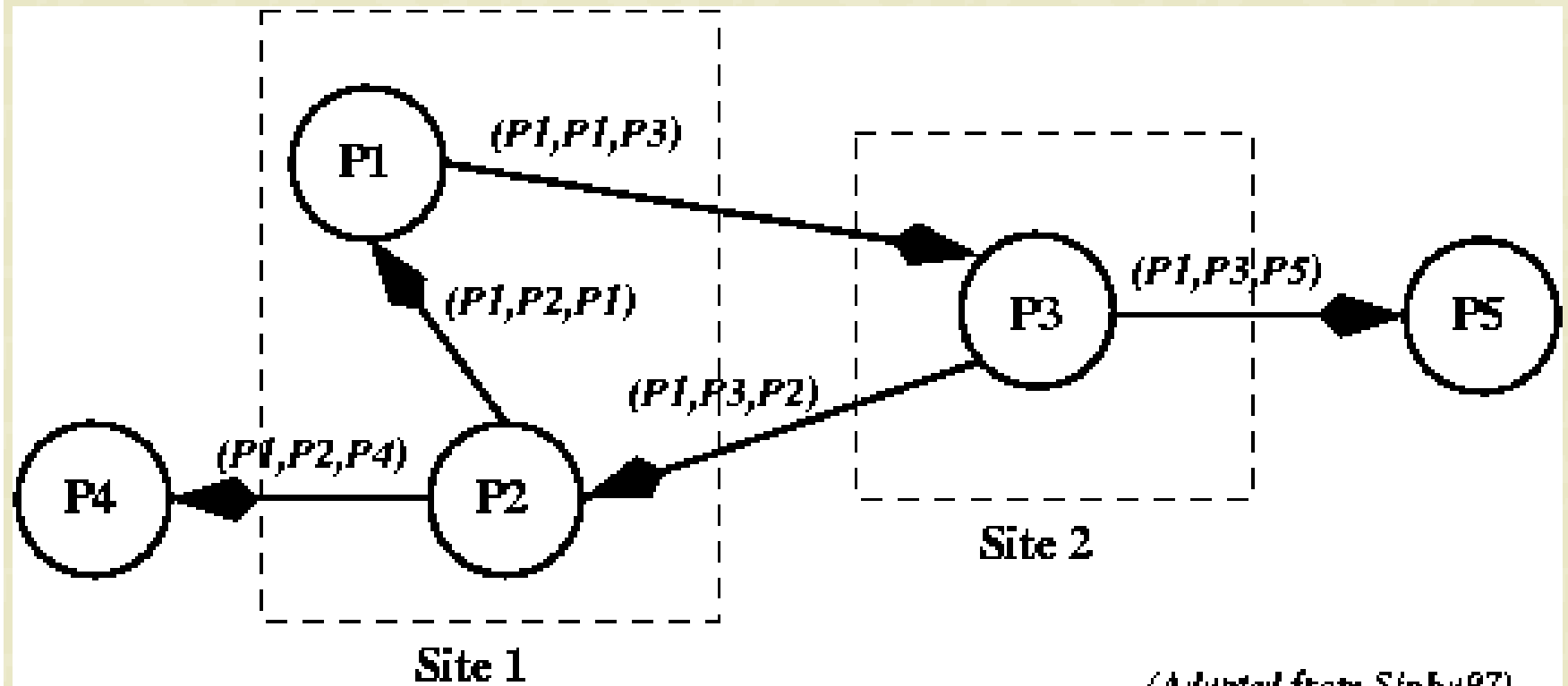
else for all P_m and P_n such that

(i) P_k is locally dependent upon P_m , and

(ii) P_m is waiting on P_n , and

(iii) P_m and P_n are on different sites,

send *probe* (i, m, n) to the home site of P_n



(Adapted from Sinha97)

Chandy et al.'s Diffusion Computation Based Algo (OR MODEL)

- Initiate a diffusion computation for a blocked process P_i :

send query (i, i, j) to each process P_j in the

dependent set DS_i of P_i ;

// processes on which P_i is dependent

$num_i(i) := |DS_i|$; $wait_i(i) := true$

- When a blocked process P_k receives a query (i, j, k) :

if this is the engaging query for process P_k then

send query (i, k, m) to all P_m in its dependent set DS_k ;

$num_k(i) := |DS_k|$; $wait_k(i) := true$

else if $wait_k(i)$ then send a reply (i, k, j) to P_j . (assume initiator

Chandy et al.'s Algo. Contd.

- When a process P_k receives a reply (i, j, k) :

if $wait_k(i)$ then begin $num_k(i) := num_k(i) - 1$;

if $num_k(i) = 0$

if $i = k$ then *declare a deadlock*

else send *reply* (i, k, m) to the process

P_m which sent the engaging query



Meritt Michelle Algorithm

- This algorithm is used for single resource model where the system automatically detects a deadlock when it occurs.
- <https://www.cmi.ac.in/~madhavan/courses/databases10/mitchell-merritt-PODC84.pdf>

Refer to the paper here.